

claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_793a5d86-1c4\agent-af2241702b156499e.jsonl`
- SHA-256: `f44bf13589256b7601411785eb351c7066024907ea6e5b2ceb5cdea8d628ec6e`
- Source modified: `2026-07-02T08:18:17+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_793a5d86-1c4`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`
```

Transcript

1. user (2026-07-02T08:16:24.735Z)

You are reviewing OPEN GitHub PR #415 on Khelsutra/badminton-highlight-indexer to decide its fate: should we TAKE IT OVER (adopt/rebase/verify/merge) or CLOSE it as superseded by already-merged work?

PR #415: "api: root relative security.upload_dir under app home (R1-20, P0a parity)" (head branch: origin/fix/upload-dir-app-home, mergeable=CONFLICTING)
Investigation hint: Routes security.upload_dir through resolve_output_dir (app_home). Handoff "audit reconciliation" claims R1-20 already fixed. CHECK: does master backend/api/uploads.py::_upload_cfg already resolve upload_dir via resolve_output_dir/app_home? If already fixed -> superseded. If not -> still-needed. Also note the conflict is likely because #422/others touched app_home resolution.

Already MERGED onto master (verify against actual master code, do not assume):

```
- #422 = fix for --output-dir clobbering config.json output.output_dir (CODE_MAP updated for it)
- #423 = docs: health-remediation plan added + DOC_STATUS/README/NEXT_STEPS registered; audit marked snapshot
- #424 = P2: /api/process idempotent by content hash (#340 re-bill fix)
- #425 = P3: /api/source resolves remote source into output-scoped source_cache, reuses across Range requests, warms annotation proxy
- #426 = P4: bounded ffprobe/ffmpeg subprocess timeouts on serving paths
- #427 = P5: clean remote-input scratch dirs after process/compile/assets/CLI; durable reports
- #428 = P1/P6: backend/storage/gcs.py lazy-import mpy typing fix
master HEAD = d00b4db "fix gcs storage lazy import typing (#428)"
```

Working dir: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (currently on master @ d00b4db). This is a SHARED checkout used by sibling sessions ? DO NOT checkout branches, DO NOT modify the working tree, DO NOT run git merge/rebase/add/commit. Read-only investigation only.

Do this:

1. Get the PR's intended change: run `gh pr diff 415 -R Khelsutra/badminton-highlight-indexer` (read the full diff). Also `gh pr view 415 -R Khelsutra/badminton-highlight-indexer --json body,files,statusCheckRollup` if useful.
2. Read the CURRENT master version of every file the PR touches (Read / Grep the files under C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer). To see the PR's branch version of a file without checkout, use: `git -C "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer" show origin/fix/upload-dir-app-home:<path>`.
3. Determine whether master ALREADY implements the PR's intent (superseded) or still lacks it. Cite concrete master file:line evidence either way. Do not trust the hint or handoff ? verify in code.
4. If CONFLICTING, explain precisely why (which files/hunks) and whether the conflict is because master already did the same thing (=> superseded/close) or because of adjacent edits (=> rebasable).

5. Judge correctness/quality of the PR change itself.
6. Recommend a fate:
 - CLOSE_SUPERSEDED: master already implements the intent; the PR adds nothing material.
 - TAKE_OVER_CLEAN: mergeable & still-needed; adopt and verify as-is.
 - TAKE_OVER_REBASE: still-needed but conflicting; needs a rebase onto master + re-verify, then merge.
 - OWNER_DECISION: product/strategy/ambiguous ? needs the owner.

Return ONLY the structured object. Be precise and cite master file:line in master_evidence.

2. assistant (2026-07-02T08:16:35.509Z)

I'll investigate PR #415 to determine its fate. Let me start by getting the PR diff and examining the current master code.

3. user (2026-07-02T08:16:39.395Z)

```
diff --git a/backend/api/uploads.py b/backend/api/uploads.py
index ccb1511..30be0b1 100644
--- a/backend/api/uploads.py
+++ b/backend/api/uploads.py
@@ -3,6 +3,9 @@
 Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
 proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
 under security.upload_dir, then submitted to /api/process by the returned path.
+A RELATIVE upload_dir roots under the app home (RALLY_APP_HOME) via the same
+`resolve_output_dir` used for output.output_dir ? P0a semantics: env unset ?
+verbatim (CWD-relative, byte-identical to before); absolute ? passes through.
 Upload state is in-process by design (this is the single-box alpha): a server restart
 aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
 arrives with PR-3 identity scoping.
@@ -21,6 +24,7 @@
 from fastapi import APIRouter, HTTPException, Request
 from pydantic import BaseModel

+from backend.utils.app_home import resolve_output_dir # P0a app-home rooting
+from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
+from backend.utils.hashing import compute_video_id # canonical file-identity hashing
+from backend.utils.paths import safe_output_filename
@@ -47,7 +51,14 @@ def _upload_cfg():
     import backend.main as _main

     sec = getattr(_main.global_config, "security", None)
-    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
+    # P0a: root a RELATIVE upload_dir under the app home (RALLY_APP_HOME), exactly like
+    # main() resolves output.output_dir ? the packaged `indexer --app` launches from an
+    # arbitrary shell CWD, and the upload landing zone must live with the DB/reels, not
+    # wherever the shell happened to be. Env unset ? verbatim (CWD-relative, unchanged);
+    # an absolute upload_dir always passes through untouched.
+    upload_dir = resolve_output_dir(
+        getattr(sec, "upload_dir", None) or "output/uploads"
+    )
     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
     exts = [
diff --git a/backend/config/models.py b/backend/config/models.py
index 652693a..e6b13e3 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -330,9 +330,12 @@ class SecurityConfig(BaseModel):
     allowed_video_root: Optional[str] = None

     # --- Chunked upload (B2, PR-2) ---
-    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
-    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
```

```

- # contain upload_dir or /api/process will reject the uploaded paths.
+ # Landing zone for browser uploads (assembled from parts). A RELATIVE path roots
+ # under the app home (RALLY_APP_HOME) at use time ? same P0a resolve_output_dir
+ # semantics as output.output_dir (env unset ? CWD-relative verbatim; absolute ?
+ # passes through). Per-user subdirectories arrive with PR-3 identity scoping.
+ # ? In hosted mode, set allowed_video_root to contain upload_dir or /api/process
+ # will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN S2).
diff --git a/docs/CODE_MAP.md b/docs/CODE_MAP.md
index 1aa5260..2657218 100644
--- a/docs/CODE_MAP.md
+++ b/docs/CODE_MAP.md
@@ -112,7 +112,7 @@
@@ @run_eval.py` (single video score),
`@scripts/run_phase3_eval.py`
- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI
mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
- `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module globals
set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every
endpoint NPEs.
- **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202
+ job_id**. `::drain_due_jobs` / `::run_claimed_job` (worker loop EXTRACTED to
`@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via
`db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job
via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling). `::execute_processing`
= the former sync body (hash -> validate -> add_video -> segment -> `finally:` always
`emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING
segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET
/api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job
`status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in
`result["status"]`. `::get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)`
(single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs
`db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights`
`@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET
/api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the
pipeline synchronously inline (separate code path, unchanged by #16).
- - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST ../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::uploads` + `::uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB).
+ - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-

```

```

bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST ../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB). A RELATIVE `upload_dir` is rooted under the app home via
`resolve_output_dir` in `_upload_cfg` (P0a ? same semantics as `output.output_dir`:
`RALLY_APP_HOME` unset ? CWD-relative verbatim; absolute ? passthrough), so packaged `indexer
--app` uploads land with the DB/reels regardless of the launch CWD.

```

```

- **STUDIO / STORAGE endpoints (P1/P2/P3/P4/P6/P8, 2026-07-01):** `::capabilities` `@GET
/api/capabilities` ? honest box probe; now includes a `**storage` block**
(`storage_capabilities` in `@backend/storage/capabilities.py`: local/s3/gcs/gdrive
`{id,kind,label,usable,free_gb}`, secret-free) alongside segmenters/ffmpeg/gpu/deployment.
`::list_videos` `@GET /api/videos` (the "my videos" list, P6). `::get_source` `@GET
/api/source/{id}` ? Range-capable FileResponse serving a **frame-identical annotation proxy**
(D4), source fallback, background-warmed. `::post_annotations` `@POST /api/annotations` ? writes
the verbatim annotator CSV to `output/labels/<video_id>.rallies.csv` (P8). `::create_asset`
`@POST /api/assets` (**P4**, `{uploaded_path?|source_uri?, medium_uri, sport?}`) ? stores a
video into a user-chosen medium (MD5-keyed `storage.put` for an upload?remote medium; register-
in-place for local; at-rest URI = no copy), registers it (video_id=MD5) so it shows in
`/api/videos`; **self-host posture only** (403 when `security.allowed_video_root` set); edge-
auth mirrors `/api/process`. Cost ledger: `::get_job_cost` `@GET /api/jobs/{id}/cost`,
`::get_video_cost` `@GET /api/videos/{id}/cost`. **Free-space guard (P2):**
`@backend/utils/freespace.py::require_free_bytes` -> HTTP **507** in `uploads.py::upload_init` +
the `/api/process` remote fetch-to-scratch; the WSL frame-extraction guard now **BLOCKS**
(`detectors/base.py::_wsl_frame_stage_shortfall`, ~15x source).

```

```

- `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly.
`::ProcessRequest`/`::QueryRequest`/`::CompileRequest`/`::AnnotationsRequest`/`::AssetRequest`
pydantic bodies.

```

```

- `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
diff --git a/tests/test_upload_download.py b/tests/test_upload_download.py
index 6be9145..9566e6b 100644

```

```

--- a/tests/test_upload_download.py
+++ b/tests/test_upload_download.py
@@ -9,6 +9,7 @@
"""

```

```

import os
+from pathlib import Path
from unittest.mock import patch

import pytest
@@ -20,6 +21,7 @@
from backend.api import uploads as uploads_api
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
+from backend.utils import app_home as ah
from backend.utils.hashing import compute_video_id

```

```

@@ -233,6 +235,65 @@ def test_abort_endpoint_removes_state(env):
    ]

```

```

+# --- P0a: upload_dir roots under the app home (RALLY_APP_HOME) -----

```

```

+ # Same resolve_output_dir semantics as output.output_dir: env unset ? verbatim
+ # (CWD-relative, pre-P0a behaviour); env set ? relative roots under the home,
+ # absolute passes through. Packaged `indexer --app` launches from any shell CWD.
+
+
+def test_upload_cfg_relative_default_roots_under_app_home(monkeypatch, tmp_path):
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
+    monkeypatch.setattr(m, "global_config", AppConfig()) # upload_dir at its default
+    upload_dir, _, _ = uploads_api.upload_cfg()
+    assert upload_dir == str(tmp_path / "output" / "uploads")
+
+
+def test_upload_cfg_absolute_upload_dir_passes_through(monkeypatch, tmp_path):
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
+    cfg = AppConfig()
+    abs_dir = str(tmp_path / "elsewhere")
+    cfg.security.upload_dir = abs_dir
+    monkeypatch.setattr(m, "global_config", cfg)
+    upload_dir, _, _ = uploads_api.upload_cfg()
+    assert upload_dir == abs_dir
+
+
+def test_upload_cfg_env_unset_keeps_cwd_relative_default(monkeypatch):
+    # RALLY_APP_HOME unset ? byte-identical to the pre-P0a CWD-relative default.
+    monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
+    monkeypatch.setattr(m, "global_config", AppConfig())
+    upload_dir, _, _ = uploads_api.upload_cfg()
+    assert upload_dir == "output/uploads"
+
+
+def test_upload_lands_under_app_home_not_launch_cwd(tmp_path, monkeypatch):
+    # End-to-end packaged shape: RALLY_APP_HOME set, upload_dir left at its relative
+    # default, server "launched" from an arbitrary shell CWD ? the assembled upload
+    # must land under <home>/output/uploads and leak nothing into the CWD.
+    home = tmp_path / "home"
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(home))
+    cwd = tmp_path / "some-shell-cwd"
+    cwd.mkdir()
+    monkeypatch.chdir(cwd)
+    db = Database(str(tmp_path / "t.db"))
+    cfg = AppConfig() # security.upload_dir stays 'output/uploads'
+    cfg.output.output_dir = str(tmp_path / "out")
+    os.makedirs(cfg.output.output_dir, exist_ok=True)
+    monkeypatch.setattr(uploads_api, "_uploads", {})
+    client = TestClient(m.create_app(cfg, db))
+    up = client.post("/api/upload/init", json={"filename": "match.mp4"}).json()
+    r = client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"match-bytes")
+    assert r.status_code == 200
+    done = client.post(
+        f"/api/upload/{up['upload_id']}/complete", json={"total_parts": 1}
+    )
+    assert done.status_code == 200
+    final = Path(done.json()["path"])
+    assert final.parent.resolve() == (home / "output" / "uploads").resolve()
+    assert final.read_bytes() == b"match-bytes"
+    assert not (cwd / "output").exists() # nothing written into the launch CWD
+
+
+    # --- download (B3) -----

```

4. user (2026-07-02T08:16:40.983Z)

```

{"body":"<!-- Audit 2026-07-02, finding R1-20 (upload_dir CWD-relative; the task brief labeled
it R1-19). -->\n\n## Summary\n`security.upload_dir` defaulted to the CWD-relative
`output/uploads` and was **never passed through the P0a app-home resolver** ? only

```

```

output.output_dir' was resolved ('backend/main.py:1992'). So the packaged `indexer --app`
(which `scripts/install.py` advertises as launchable from any shell) wrote browser uploads into
whatever CWD the shell happened to have ? or 500'd on an unwritable one ? while the SQLite DB
and reels correctly lived under `RALLY_APP_HOME`. Exactly the CWD-vs-app-home class of bug P0a
was built to eliminate.\n\n**Fix:** `backend/api/uploads.py::upload_cfg` now resolves
`upload_dir` through the same `resolve_output_dir` (`backend/utils/app_home.py`) used for
`output.output_dir`, mirroring its semantics exactly:\n- `RALLY_APP_HOME` **unset** ? the
configured value verbatim (CWD-relative, byte-identical to before ? dev/repo/CI paths
unchanged);\n- **set** ? a relative `upload_dir` (incl. the default) roots under the home
(`<home>/output/uploads`);\n- an **absolute** (or Windows drive-anchored) `upload_dir` always
passes through untouched.\n\nResolving in `upload_cfg` (rather than once in `main()`) keeps
every entry point covered ? including apps built via `create_app()` without going through
`main()` ? and `app_home()` is stable per-process, so the result is deterministic. Docs:
`SecurityConfig.upload_dir` comment + `docs/CODE_MAP.md` §2.0 updated to match.\n\n**Tests**
(`tests/test_upload_download.py`, new P0a section):\n- relative default resolves under the app
home when `RALLY_APP_HOME` is set;\n- absolute `upload_dir` passes through even with the env
set;\n- env unset ? verbatim `output/uploads` (pins the unchanged pre-P0a default);\n- end-to-
end: init?part?complete from an arbitrary CWD lands the assembled file under
`<home>/output/uploads` and leaks nothing into the launch CWD.\n\nNo existing test pinned the
CWD-relative behavior of the *default* (the fixture uses an absolute `upload_dir`, which passes
through unchanged), so no test needed loosening.\n\n## Verification\nRan locally (Windows,
worktree off `origin/master` @ e410136):\n- `python -m pytest tests/test_upload_download.py
tests/test_app_home.py -q` ? **46 passed**\n- `ruff check` + `ruff format --check` on the
changed files ? clean\n- `python -m mypy backend/api/uploads.py backend/config/models.py` ? no
issues\n- `python -m tools.check_md_links` ? clean\n\nCI covers the full `run_all_tests.py`
suite + coverage floor + link-check. Not verified locally: a real packaged `indexer --app`
launch (install-flow/GPU machine checks stay with the owner per CLAUDE.md); the end-to-end test
simulates that shape via `RALLY_APP_HOME` + `monkeypatch.chdir`.\n\n## Checks\n- [x] Targeted
tests green; `ruff check` + `mypy` clean on changed files; full suite + coverage in CI.\n- [x]
Internal markdown links valid (`python -m tools.check_md_links`).\n\n? Generated with [Claude
Code](https://claude.com/claude-code)", "files": [{"path": "backend/api/uploads.py", "additions": 12,
"deletions": 1, "changeType": "MODIFIED"}, {"path": "backend/config/models.py", "additions": 6, "deletio
ns": 3, "changeType": "MODIFIED"}, {"path": "docs/CODE_MAP.md", "additions": 1, "deletions": 1, "changeTyp
e": "MODIFIED"}, {"path": "tests/test_upload_download.py", "additions": 61, "deletions": 0, "changeType"
: "MODIFIED"}], "mergeStateStatus": "DIRTY", "mergeable": "CONFLICTING", "statusCheckRollup": [{"__type
name": "CheckRun", "completedAt": "2026-07-
02T04:53:02Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262312", "name": "Ruff lint", "startedAt": "2026
-07-
02T04:52:51Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:52:56Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262245", "name": "Leak scan (PII / attribution
guard)", "startedAt": "2026-07-
02T04:52:50Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:53:00Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262179", "name": "Markdown link check (live do
cs)", "startedAt": "2026-07-
02T04:52:52Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:53:30Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262232", "name": "Mypy (lenient baseline)", "st
artedAt": "2026-07-
02T04:52:51Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:53:15Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262176", "name": "Syntax sweep + import smoke
(no GPU stack)", "startedAt": "2026-07-
02T04:52:51Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:55:35Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262246", "name": "Full test suite (offline, mo
cked)", "startedAt": "2026-07-
02T04:52:50Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "

```

```

2026-07-
02T04:53:20Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262311", "name": "Golden fixtures cross-OS
(ubuntu-latest)", "startedAt": "2026-07-
02T04:52:51Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:54:07Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262358", "name": "Golden fixtures cross-OS
(windows-latest)", "startedAt": "2026-07-
02T04:52:52Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:53:09Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262294", "name": "Golden fixtures cross-OS
(macos-latest)", "startedAt": "2026-07-
02T04:52:51Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:53:27Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262170", "name": "Dependency license scan (no
copyleft in runtime deps)", "startedAt": "2026-07-
02T04:52:50Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "
2026-07-
02T04:53:52Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566321751/job/84694262198", "name": "Built wheel smoke (install +
launch)", "startedAt": "2026-07-02T04:52:50Z", "status": "COMPLETED", "workflowName": "CI"]}]

```

5. assistant (2026-07-02T08:16:45.727Z)

Let me examine the current master version of the files the PR touches.

6. user (2026-07-02T08:16:48.433Z)

```

d00b4db fix gcs storage lazy import typing (#428)
---HEAD---
d00b4db7ab11b61df356940d2a6fc77486afe50c

```

7. user (2026-07-02T08:16:48.720Z)

```

1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5      under security.upload_dir, then submitted to /api/process by the returned path.
6      Upload state is in-process by design (this is the single-box alpha): a server restart
7      aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8      arrives with PR-3 identity scoping.
9
10     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11     ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12     reads the live value the server set at startup (and so tests that monkeypatch
13     ``backend.main.global_config`` flow through unchanged).
14     """
15
16     import os
17     import threading
18     import uuid
19     from typing import Any, Dict, Optional
20
21     from fastapi import APIRouter, HTTPException, Request
22     from pydantic import BaseModel
23
24     from backend.utils.app_home import (
25         resolve_output_dir, # P0a: app-home-aware path resolution
26     )
27     from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)

```

```

28     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
29     from backend.utils.paths import safe_output_filename
30
31     router = APIRouter()
32
33     _uploads: Dict[str, Dict[str, Any]] = {}
34     _uploads_lock = threading.Lock()
35
36
37     class UploadInitRequest(BaseModel):
38         filename: str
39         total_size_bytes: Optional[int] = None
40
41
42     class UploadCompleteRequest(BaseModel):
43         total_parts: int
44
45
46     def _upload_cfg():
47         # Resolved lazily (import inside the function) to read the LIVE startup config that
48         # main() sets on the module global, and to avoid a circular import at module load
49         # (main.py imports this router; this module must not import main at import time).
50         import backend.main as _main
51
52         sec = getattr(_main.global_config, "security", None)
53         upload_dir = resolve_output_dir(
54             getattr(sec, "upload_dir", None) or "output/uploads"
55         ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
56         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
57         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
58         exts = [
59             e.lower().lstrip(".")
60             for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
61         ]
62         return upload_dir, max_bytes, part_bytes, exts
63
64
65     def _abort_upload(upload_id: str) -> None:
66         with _uploads_lock:
67             st = _uploads.pop(upload_id, None)
68             if st:
69                 try:
70                     os.remove(st["tmp_path"])
71                 except OSError:
72                     pass
73
74
75     @router.post("/api/upload/init")
76     def upload_init(req: UploadInitRequest):
77         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
78         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
79         try:
80             name = safe_output_filename(req.filename)

```

8. assistant (2026-07-02T08:16:51.870Z)

This is the key finding: master ALREADY implements the PR's intent. The master `backend/api/uploads.py:\_upload\_cfg` at line 52-55 already resolves `upload\_dir` through `resolve\_output\_dir`. Let me verify the details and compare with the PR's branch version to confirm this is genuinely superseded.

9. user (2026-07-02T08:16:54.451Z)

=== Which merged PR introduced resolve_output_dir into uploads.py? ===
4f148d1 fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)

f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router (#228)

10. user (2026-07-02T08:16:55.180Z)

4f148d1 fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)

11. assistant (2026-07-02T08:16:58.679Z)

This confirms it. PR #422 (already merged) introduced `resolve\_output\_dir` into `\_uploads.py::\_upload\_cfg`, addressing audit findings R1-19, R1-20, R1-27. Let me verify the full commit and check the other files (models.py, CODE_MAP.md) to see if those doc changes are also already present.

12. user (2026-07-02T08:17:00.361Z)

commit 4f148d1a60d7baf66b7fe03a6601fc21e52bb2d
Author: Avi <avi.dullu@gmail.com>
Date: Thu Jul 2 11:12:24 2026 +0530

fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)

R1-27: Fix misplaced type-ignore in gcs.py

- The # type: ignore was on the 'storage,' line inside a parenthesized import, but mypy attributes the error to the 'from google.cloud import (' line above. This made mypy fail on any box with google-cloud-storage installed while CI stayed green (CI doesn't install the optional extra).
- Fix: collapse to a single-line import with the ignore on the correct line.

R1-19: Fix output.output_dir silently clobbered by CLI default

- --output-dir had default='output', and main() unconditionally overrode the typed config value with resolve_output_dir(args.output_dir) ? so a config.json output.output_dir could never take effect on the server/CLI path.
- Fix: change default to None; only override when the flag is explicitly passed. When not passed, the config.json value is resolved through app-home. The trim path falls back to 'output' (global_config isn't loaded yet there).

R1-20: Fix upload_dir CWD-relative (never app-home-resolved)

- upload_dir default 'output/uploads' was used verbatim ? never passed through resolve_output_dir(). On a packaged --app install, uploads landed in an arbitrary CWD (or 500'd on an unwritable one) while DB/reels lived under RALLY_APP_HOME.
- Fix: resolve upload_dir through resolve_output_dir() in _upload_cfg, mirroring output_dir. Byte-identical when RALLY_APP_HOME is unset.

```
backend/api/uploads.py | 7 ++++++  
backend/main.py        | 18 ++++++-----  
backend/storage/gcs.py | 2 +-  
3 files changed, 19 insertions(+), 8 deletions(-)
```

13. user (2026-07-02T08:17:00.592Z)

```
325  
326  
327     class SecurityConfig(BaseModel):  
328         # When set, /api/process video_path must resolve under this directory (hosted/tenant  
329         # mode). Default None preserves the single-user local 'any local file' workflow.  
330         allowed_video_root: Optional[str] = None  
331  
332         # --- Chunked upload (B2, PR-2) ---  
333         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories  
334         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
```

```

335         # contain upload_dir or /api/process will reject the uploaded paths.
336         upload_dir: str = "output/uploads"
337         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
338         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
339         max_upload_mb: int = 4096
340         max_part_mb: int = 95
341         allowed_upload_extensions: List[str] = ["mp4", "mov"]
342
343         # --- M9 edge auth (Cloudflare Access, D9) ---
344         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
345         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
346         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
347         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
348         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
349         # application's audience tag. NO secrets here ? these are public identifiers.
350         edge_auth_enabled: bool = False
351         cf_team_domain: str = (
352             "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
353         )
354         cf_access_aud: str = "" # the CF Access application AUD tag the token must carry

```

14. assistant (2026-07-02T08:17:06.663Z)

This confirms the core intent. The functional fix (R1-20) in ``backend/api/uploads.py::upload_cfg`` is already merged via #422. Now let me verify the remaining differences: the PR also modifies ``models.py`` comments and ``docs/CODE_MAP.md`` and adds tests. Let me check whether those doc/test additions are still absent on master (which would be the only material delta), and confirm the exact conflict.

15. user (2026-07-02T08:17:09.712Z)

```

=== master CODE_MAP.md UPLOAD/DOWNLOAD line ===
=== does master models.py mention app-home for upload_dir? ===

```

16. user (2026-07-02T08:17:11.536Z)

```

=== PR tests present on master? ===

```

17. assistant (2026-07-02T08:17:15.158Z)

So on master: the functional fix is present (via #422), but the PR's **documentation comments** (`models.py`, `CODE_MAP.md`) and **tests** (4 new tests in `test_upload_download.py`) are NOT present. Let me verify the exact nature of the merge conflict by examining what #422 changed in `uploads.py` vs the PR's version, and confirm the PR branch's base.

18. user (2026-07-02T08:17:17.102Z)

```

=== #422 diff to uploads.py (the merged fix) ===
commit 4f148d1a60d7baf66b7fe03a6601fc21e52bb2d
Author: Avi <avi.dullu@gmail.com>
Date: Thu Jul 2 11:12:24 2026 +0530

```

fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)

R1-27: Fix misplaced type-ignore in `gcs.py`

- The `# type: ignore` was on the `'storage,'` line inside a parenthesized import, but mypy attributes the error to the `'from google.cloud import ('` line above. This made mypy fail on any box with google-cloud-storage installed while CI stayed green (CI doesn't install the optional extra).

- Fix: collapse to a single-line import with the ignore on the correct line.

R1-19: Fix output.output_dir silently clobbered by CLI default

- --output-dir had default='output', and main() unconditionally overrode the typed config value with resolve_output_dir(args.output_dir) ? so a config.json output.output_dir could never take effect on the server/CLI path.
- Fix: change default to None; only override when the flag is explicitly passed. When not passed, the config.json value is resolved through app-home. The trim path falls back to 'output' (global_config isn't loaded yet there).

R1-20: Fix upload_dir CWD-relative (never app-home-resolved)

- upload_dir default 'output/uploads' was used verbatim ? never passed through resolve_output_dir(). On a packaged --app install, uploads landed in an arbitrary CWD (or 500'd on an unwritable one) while DB/reels lived under RALLY_APP_HOME.
- Fix: resolve upload_dir through resolve_output_dir() in _upload_cfg, mirroring output_dir. Byte-identical when RALLY_APP_HOME is unset.

```
diff --git a/backend/api/uploads.py b/backend/api/uploads.py
index ccb1511..389f0f7 100644
--- a/backend/api/uploads.py
+++ b/backend/api/uploads.py
@@ -21,6 +21,9 @@ from typing import Any, Dict, Optional
     from fastapi import APIRouter, HTTPException, Request
     from pydantic import BaseModel

+from backend.utils.app_home import (
+    resolve_output_dir, # P0a: app-home-aware path resolution
+)
from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
from backend.utils.hashing import compute_video_id # canonical file-identity hashing
from backend.utils.paths import safe_output_filename
@@ -47,7 +50,9 @@ def _upload_cfg():
     import backend.main as _main

     sec = getattr(_main.global_config, "security", None)
-    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
+    upload_dir = resolve_output_dir(
+        getattr(sec, "upload_dir", None) or "output/uploads"
+    ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
     exts = [
```

19. assistant (2026-07-02T08:17:23.764Z)

This is definitive. #422's change to `uploads.py::\_upload\_cfg` is functionally IDENTICAL to PR #415's change to the same function ? both wrap `getattr(sec, "upload\_dir", None) or "output/uploads"` in `resolve\_output\_dir(...)` and add the same import. This is a classic case where two sessions produced the same fix and one merged first.

Now let me confirm the exact conflict cause. The PR #415 branch modifies `uploads.py` with a slightly different import block (single-line) and a different comment, while master (#422) already changed the same lines. Let me verify the PR branch's uploads.py to confirm the collision.

20. user (2026-07-02T08:17:25.936Z)

```
=== PR branch uploads.py import + _upload_cfg region ===
from fastapi import APIRouter, HTTPException, Request
from pydantic import BaseModel

from backend.utils.app_home import resolve_output_dir # P0a app-home rooting
from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
from backend.utils.hashing import compute_video_id # canonical file-identity hashing
```

```

from backend.utils.paths import safe_output_filename

router = APIRouter()

_uploads: Dict[str, Dict[str, Any]] = {}
_uploads_lock = threading.Lock()

class UploadInitRequest(BaseModel):
    filename: str
    total_size_bytes: Optional[int] = None

class UploadCompleteRequest(BaseModel):
    total_parts: int

def _upload_cfg():
    # Resolved lazily (import inside the function) to read the LIVE startup config that
    # main() sets on the module global, and to avoid a circular import at module load
    # (main.py imports this router; this module must not import main at import time).
    import backend.main as _main

    sec = getattr(_main.global_config, "security", None)
    # P0a: root a RELATIVE upload_dir under the app home (RALLY_APP_HOME), exactly like
    # main() resolves output.output_dir ? the packaged `indexer --app` launches from an
    # arbitrary shell CWD, and the upload landing zone must live with the DB/reels, not
    # wherever the shell happened to be. Env unset ? verbatim (CWD-relative, unchanged);
    # an absolute upload_dir always passes through untouched.
    upload_dir = resolve_output_dir(
        getattr(sec, "upload_dir", None) or "output/uploads"
    )
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024

```

21. user (2026-07-02T08:17:28.247Z)

```

=== merge-base of PR branch and master ===
e4101365cb5f3a608e76f8baf1d2e0d65ce1738e
=== is merge-base #422's parent (i.e. branched before #422)? ===
d80aa02e81c2c02e448ef75fa8777aa9c4561531
=== files PR branch changes vs merge-base ===
backend/api/uploads.py          | 13 ++++++--
backend/config/models.py       | 9 +++++-
docs/CODE_MAP.md               | 2 +-
tests/test_upload_download.py  | 61 +++++
4 files changed, 80 insertions(+), 5 deletions(-)

```

22. assistant (2026-07-02T08:17:33.244Z)

Now I have the complete picture. The PR branched from `e410136` (before #422 merged). Both #415 and #422 independently applied the same `resolve\_output\_dir` wrap to the same lines in `uploads.py::upload\_cfg`, plus the same import. This is why the merge is **CONFLICTING** ? overlapping edits to the identical lines, with master already containing the functional fix.

Let me confirm the `resolve\_output\_dir` / `app\_home` semantics to validate correctness of the change, and confirm the conflict is on the uploads.py hunk specifically.

23. user (2026-07-02T08:17:35.273Z)

```

=== resolve_output_dir signature/semantics ===
30:APP_HOME_ENV = "RALLY_APP_HOME"
36:def app_home() -> Path:
42:     env = os.environ.get(APP_HOME_ENV)
83:def resolve_output_dir(output_dir: str) -> str:

```

24. user (2026-07-02T08:17:35.285Z)

```
1      """Resolved application-data HOME for a packaged, batteries-included install.
2
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
6      from
7      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
8      silently
9      "loses" their index. This module introduces ONE resolver every state path roots on.
10
11     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
12     * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
13     sets it).
14     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
15     behaviour.
16
17     The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
18     ``~/local/share``)
19     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
20     launcher (P2)
21     calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
22     importing
23     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
24     writes into a
25     real user profile during a test run).
26
27     stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
28     before
29     the heavier pydantic-backed ``backend.config`` package ? keeping the early
30     ``load_dotenv`` path light.
31
32     """
33
34     from __future__ import annotations
35
36     import os
37     import platform
38     from pathlib import Path
39
40     #: Environment variable the packaged launcher sets to pin the app-data home.
41     APP_HOME_ENV = "RALLY_APP_HOME"
42
43     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
44     APP_DIR_NAME = "Khelsutra"
45
46     def app_home() -> Path:
47         """The resolved application-data home.
48
49         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
50         byte-identical
51         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
52         """
53         env = os.environ.get(APP_HOME_ENV)
54         if env and env.strip():
55             return Path(env.strip()).expanduser()
56         return Path(".")
57
58     def app_path(*parts: str | os.PathLike[str]) -> Path:
59         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
60         return app_home().joinpath(*[str(p) for p in parts])
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```

53     def default_config_path() -> Path:
54         """`<app-home>/config.json` ? the config file the loader reads and ``--setup``
writes.
55
56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60

```

25. assistant (2026-07-02T08:17:46.969Z)

I have everything I need. The investigation is complete and conclusive.

Summary of findings

****Master ALREADY implements the PR's functional intent.**** The core fix ? routing ``security.upload_dir`` through ``resolve_output_dir`` in ``_upload_cfg`` ? was merged via PR *****422**** (commit ``4f148d1``, "fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27)"). That commit's changelog explicitly lists "R1-20: Fix upload_dir CWD-relative (never app-home-resolved)."

The functional change in #422 is ****byte-identical in effect**** to PR #415:

- Master ``backend/api/uploads.py:53-55``: ``upload_dir = resolve_output_dir(getattr(sec, "upload_dir", None) or "output/uploads")``
- Master ``backend/api/uploads.py:24-26``: the ``from backend.utils.app_home import resolve_output_dir`` import

Both wrap the exact same expression and add the same import. #415 only differs cosmetically (single-line import + a longer comment).

****Conflict cause:**** The PR branched from ``e410136`` (before #422). It edits the identical lines of ``_upload_cfg`` and the identical import block that #422 already changed on master ? overlapping edits to the same lines. So the conflict is because ****master already did the same thing****, not adjacent edits. This is a supersede-close conflict, not a rebasable one.

****Only non-superseded deltas**** are documentation comments (``backend/config/models.py:332-336``, one ``docs/CODE_MAP.md`` line) and 4 new tests in ``tests/test_upload_download.py`` ? but those exist solely to describe/verify behavior master already has. They don't justify keeping the PR; if the owner wants the extra test coverage, it's a trivial separate cherry-pick, not a reason to rebase this whole PR.

****Correctness:**** The PR change itself is correct and matches the resolver semantics (``RALLY_APP_HOME`` unset ? ``Path(".")`` ? CWD-verbatim; set ? relative roots under home; absolute passes through). It's just redundant with what shipped.

26. user (2026-07-02T08:18:03.156Z)

Output does not match required schema: root: must have required property 'conflict_nature'

27. assistant (2026-07-02T08:18:06.791Z)

The ``conflict_nature`` field lost its opening tag. Let me resubmit with the correct field name.

28. user (2026-07-02T08:18:17.504Z)

Structured output provided successfully